

Spring Boot JWT Authentication

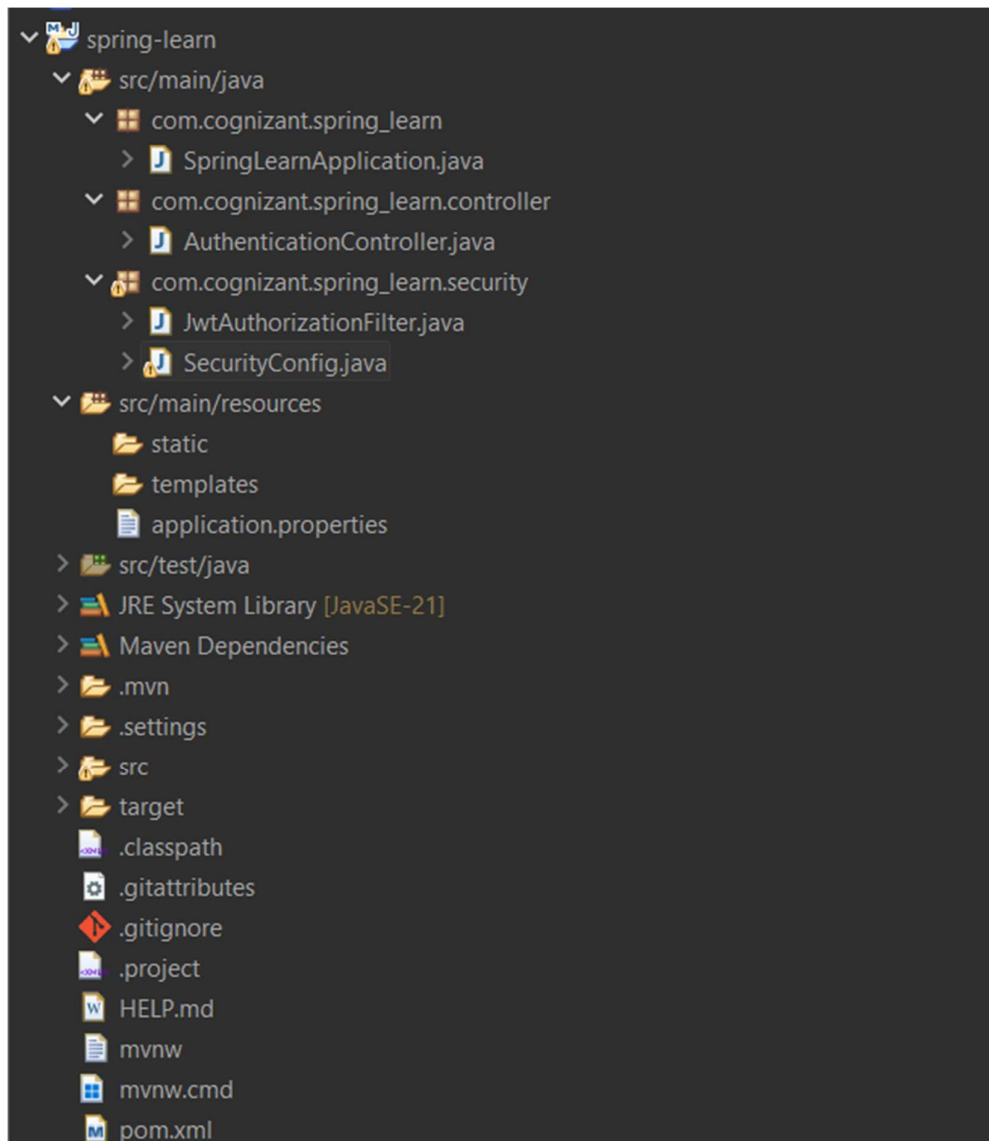
Task Objective:

Build an authentication service using Spring Boot + Spring Security + JWT that:

- Accepts username/password via Basic Auth
- Returns a signed JWT token
- Protects other REST endpoints using the JWT token

Project Setup

Project Name: spring-learn



Dependencies in pom.xml

```
<dependencies>
    <!-- Spring Boot Web -->
    <dependency>

    <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-
web</artifactId>
    </dependency>

    <!-- Spring Security -->
    <dependency>

    <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-
security</artifactId>
    </dependency>

    <!-- JWT -->
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt</artifactId>
        <version>0.9.0</version>
    </dependency>

    <!-- Fix for Java 11+ (JAXB) -->
    <dependency>
        <groupId>javax.xml.bind</groupId>
        <artifactId>jaxb-api</artifactId>
        <version>2.3.1</version>
    </dependency>
</dependencies>
```

Application Configuration

File: application.properties
server.port=8080

Spring Boot Entry Point

File: SpringLearnApplication.java
package com.cognizant.spring_learn;

```

import
org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringLearnApplication {
    public static void main(String[] args) {

SpringApplication.run(SpringLearnApplication.class, args);
    }
}

```

Security Configuration

File: SecurityConfig.java

package com.cognizant.spring_learn.security;

```

import
org.springframework.context.annotation.Bean;
import
org.springframework.context.annotation.Configuration;
import
org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.builders.AuthenticationManagerBuilder;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;

```

```

import
org.springframework.security.crypto.password.PasswordEncoder;
import
org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager
authManager(HttpSecurity http, PasswordEncoder
encoder) throws Exception {
        AuthenticationManagerBuilder auth =
http.getSharedObject(AuthenticationManagerBuilder.class);
        auth.inMemoryAuthentication()

.withUser("user").password(encoder.encode("pwd"))
.roles("USER")
        .and()

.withUser("admin").password(encoder.encode("pwd"))
).roles("ADMIN");
        return auth.build();
    }

    @Bean
    public SecurityFilterChain
filterChain(HttpSecurity http,
AuthenticationManager authManager) throws
Exception {
        http.csrf().disable()
        .authorizeHttpRequests()

.requestMatchers("/authenticate").permitAll()
        .anyRequest().authenticated()

```

```

        .and()
        .authenticationManager(authManager)
        .addFilter(new
JwtAuthorizationFilter(authManager));

        return http.build();
    }
}

```

Authentication Controller

File: AuthenticationController.java

package com.cognizant.spring_learn.controller;

```

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.JwtBuilder;
import
org.springframework.web.bind.annotation.*;

import java.util.Base64;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;

@RestController
public class AuthenticationController {

    @GetMapping("/authenticate")
    public Map<String, String>
authenticate(@RequestHeader("Authorization")
String authHeader) {
        String user = getUser(authHeader);
        String token = generateJwt(user);

        Map<String, String> map = new
HashMap<>();
        map.put("token", token);
        return map;
    }

    private String getUser(String authHeader) {

```

```

        String encoded =
authHeader.substring(6);
        byte[] decodedBytes =
Base64.getDecoder().decode(encoded);
        return new
String(decodedBytes).split(":")[0];
    }

    private String generateJwt(String user) {
        JwtBuilder builder = Jwts.builder();
        builder.setSubject(user);
        builder.setIssuedAt(new Date());
        builder.setExpiration(new
Date(System.currentTimeMillis() + 20 * 60 *
1000));

builder.signWith(SignatureAlgorithm.HS256,
"secretkey");
        return builder.compact();
    }
}

```

JWT Filter

File: JwtAuthorizationFilter.java

package com.cognizant.spring_learn.security;

```

import io.jsonwebtoken.*;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import
org.springframework.security.authentication.Auth
enticationManager;
import
org.springframework.security.authentication.User
namePasswordAuthenticationToken;
import
org.springframework.security.core.context.Securi
tyContextHolder;

```

```

import
org.springframework.security.web.authentication.
www.BasicAuthenticationFilter;

import java.io.IOException;
import java.util.ArrayList;

public class JwtAuthorizationFilter extends
BasicAuthenticationFilter {

    public
JwtAuthorizationFilter(AuthenticationManager
authManager) {
        super(authManager);
    }

    @Override
    protected void
doFilterInternal(HttpServletRequest req,
HttpServletRequest res, FilterChain chain)
        throws IOException, ServletException
    {

        String header =
req.getHeader("Authorization");

        if (header == null ||
!header.startsWith("Bearer ")) {
            chain.doFilter(req, res);
            return;
        }

        UsernamePasswordAuthenticationToken
authentication = getAuthentication(req);

SecurityContextHolder.getContext().setAuthentica
tion(authentication);
        chain.doFilter(req, res);
    }

    private UsernamePasswordAuthenticationToken
getAuthentication(HttpServletRequest request) {
        String token =
request.getHeader("Authorization");

```

```

        if (token != null) {
            try {
                String user = Jwts.parser()

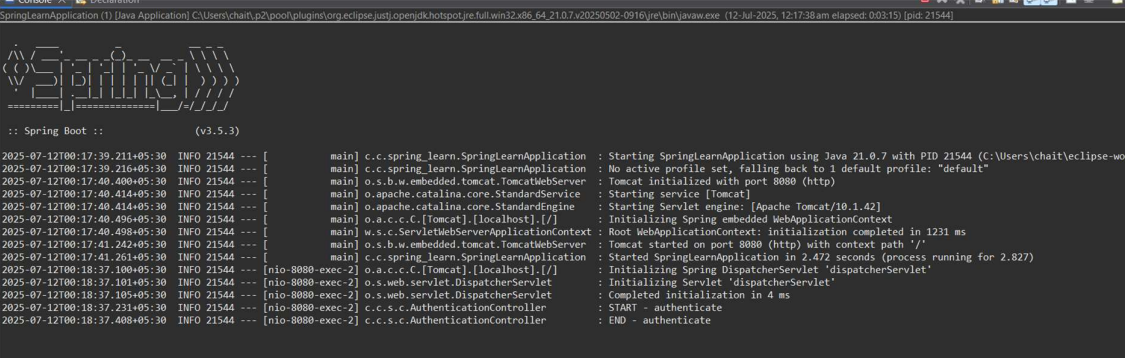
                .setSigningKey("secretkey")

                .parseClaimsJws(token.replace("Bearer ", ""))
                    .getBody()
                    .getSubject();

                if (user != null) {
                    return new
UsernamePasswordAuthenticationToken(user, null,
new ArrayList<>());
                }
            } catch (Exception e) {
                return null;
            }
        }
        return null;
    }
}

```

Console :



```

SpringLearnApplication (1) [Java Application] C:\Users\chait.p2\poo\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.21.0.7\j20250502-0916\jre\bin\java.exe (12-Jul-2025, 12:17:38 am elapsed: 0:03:15) [pid: 21544]

:: Spring Boot ::
(v3.5.3)

2025-07-12T00:17:39.211+05:30 INFO 21544 --- [main] c.c.spring_learn.SpringLearnApplication : Starting SpringLearnApplication using Java 21.0.7 with PID 21544 (C:\Users\chait.p2\poo\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.21.0.7\j20250502-0916\jre\bin\java.exe)
2025-07-12T00:17:39.216+05:30 INFO 21544 --- [main] c.c.spring_learn.SpringLearnApplication : No active profile set, falling back to 1 default profile: "default"
2025-07-12T00:17:40.400+05:30 INFO 21544 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2025-07-12T00:17:40.414+05:30 INFO 21544 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2025-07-12T00:17:40.414+05:30 INFO 21544 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.42]
2025-07-12T00:17:40.496+05:30 INFO 21544 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2025-07-12T00:17:40.498+05:30 INFO 21544 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1231 ms
2025-07-12T00:17:41.242+05:30 INFO 21544 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2025-07-12T00:17:41.261+05:30 INFO 21544 --- [main] c.c.spring_learn.SpringLearnApplication : Started SpringLearnApplication in 2.472 seconds (process running for 2.827)
2025-07-12T00:18:37.100+05:30 INFO 21544 --- [nio-8080-exec-2] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2025-07-12T00:18:37.101+05:30 INFO 21544 --- [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2025-07-12T00:18:37.105+05:30 INFO 21544 --- [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Completed initialization in 4 ms
2025-07-12T00:18:37.231+05:30 INFO 21544 --- [nio-8080-exec-2] c.c.s.c.AuthenticationController : START - authenticate
2025-07-12T00:18:37.408+05:30 INFO 21544 --- [nio-8080-exec-2] c.c.s.c.AuthenticationController : END - authenticate

```

Testing /authenticate Endpoint (Postman)

- Method: **GET**
- URL: **http://localhost:8080/authenticate**
- Authorization: Basic Auth
 - Username: user

○ Password: pwd

The screenshot shows a REST client interface with a list of requests at the top. The selected request is a GET to `http://localhost:8080/authenticate`. The **Authorization** tab is active, showing **Basic Auth** with `user` as the username and `pwd` as the password. Below this, a message states: "The authorization header will be automatically generated when you send the request. [Learn more about authorization](#)". A "Store in Vault" button is also present.

The **Body** tab at the bottom shows the response in JSON format:

```
1 {
2   "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjE5NzE3LCJleHAiOjE5NzE3NTIyNjA5MTd9.VaQECTCJkDdsXARqyBpb31S_QN2oIzEBD--136IEUM0"
3 }
```

Response details: Status: 200 OK, Time: 475 ms, Size: 475 B. A "Save Response" button is available.